SYSTEM DESIGN · BOTH PLANES, AS BUILT

AgentShield
the system, part by part

The build reference. Every deployable component of both planes, the interface it exposes, the data it reads and writes — and the map from a feature we are about to build to the parts of the system it touches.

> **What this document is for.** It is the document you keep open while building. It names each component of the synchronous and asynchronous planes, fixes the contract between them, writes down the shape of every store and message, and ends with a build map from feature to components, stores and phase. It assumes the product and the engineering architecture; it does not re-argue them.

Companion to AgentShield — Product Document & Engineering Architecture
Gate `POST /v1/orders`, before any rupee moves
Planes synchronous decision service · asynchronous workers over a bus
Version 1.0 · **Date** 27 August 2026

Prepared by Sagar Khandagre (Cloud Native Security Contributor)
Passionate about Agentic security
GitHub github.com/sagarkhandagre998

— How to read this document

A design you build from, not a design you admire. Each section names a part, its interface, and the data it owns.

This is the system design for AgentShield — the trust and risk layer that answers at `POST /v1/orders`, before any rupee moves. The product document says *what* the system decides and *why*; the engineering document says how it holds its latency. This document says what the parts are and how they connect, so that when we build a feature we know exactly which components, stores and messages it touches.

Everything rests on one split, and it is worth stating once before anything else. AgentShield runs as two planes. The SYNCHRONOUS plane is on the request — a person or an agent is waiting — and all it is allowed to do is *read keyed data and compare it*. The ASYNCHRONOUS plane is off the request; it does the slow thinking — models, embeddings, the graph — and leaves each answer behind as a plain number. The two never call each other. They meet at exactly two pieces of durable infrastructure: events go *down* to the bus, and one precomputed figure comes *up* from the feature store.

2	1	6	5
PLANES — ONE DECIDES, ONE THINKS AHEAD	STATELESS SERVICE ON THE CLOCK	ASYNCHRONOUS WORKERS OVER ONE BUS	STORES — THREE HOT, TWO OFF THE CLOCK

How the document is laid out

Part one draws the whole system in two figures — the context and the component map. Part two is the synchronous plane: the decision service, its `Evaluate` contract, and the six predicates. Part three is the asynchronous plane: the bus, the workers, and the one-directional coupling that joins them. Part four is the data design — the five stores and every core schema. Part five treats the three ML engines as services with a single serving contract. Part six is cross-cutting concerns and the build map — the table to keep open while building.

> **COLOUR CARRIES MEANING**
>
> ON THE CLOCK marks work on the synchronous path; OFF THE CLOCK marks work done ahead of time. ALLOW STEP-UP BLOCK are the three answers. A figure in a store is written by a worker and read by the request — never computed while the customer waits.

> **TWO INVARIANTS THAT NEVER BEND**
>
> Only the six predicates can block; the ML engines may only *raise* risk or ask for a step-up, never refuse on their own. And when anything is missing, stale or slow, the system fails closed to a step-up — it never guesses in order to stay fast. Every design choice below serves those two rules.

— Contents

Six parts, fifteen sections, six figures. Every section names a part of the system, its interface, and the data it owns.

> **HOW THIS DIFFERS FROM THE ENGINEERING DOCUMENT**
>
> The engineering document defends a number — the latency budget. This one draws the parts. Where the two overlap — the two planes, the stores — this document restates only enough to stand alone, then goes to the interfaces, schemas and the build map that a builder needs and the performance document did not carry.

01

PART ONE

The system in one view

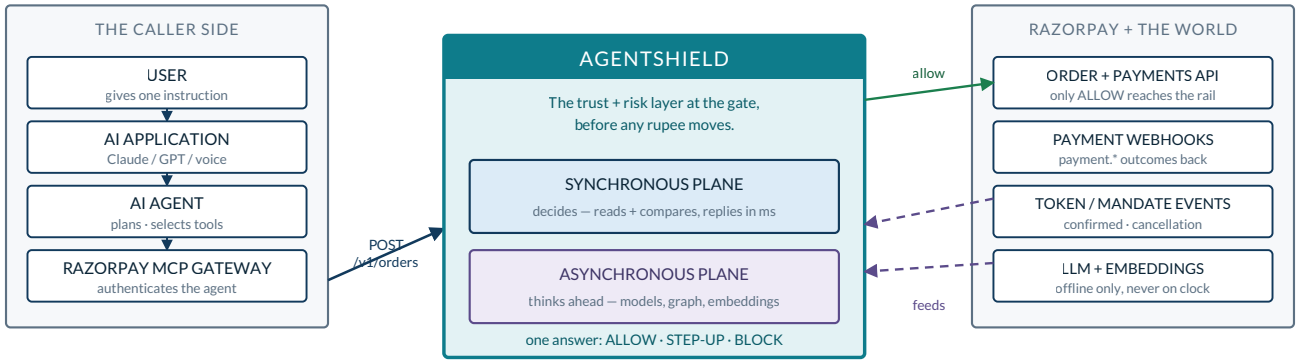
Before the parts, the whole. Two figures carry it: the context — who calls AgentShield and what it keeps off the request — and the component map, every deployable part of both planes and the one store they share. Read these two and the rest is detail.

1 System context

One box in the caller's payment chain, consulted once, before any rupee moves.

AgentShield is not the payment system and it is not the agent. It is a single decision that sits *between* them: when an agent asks Razorpay to create an order, the request is shown to AgentShield first, and AgentShield returns one of three answers — `ALLOW`, `STEP-UP` or `BLOCK`. Only `ALLOW` proceeds to `POST /v1/payments/create/recurring` and the rails. Because the gate runs *before* money moves, a wrong "no" costs a re-confirmation, not a reversal — which is what lets the whole system fail closed without fear.

SYSTEM CONTEXT — WHO CALLS AGENTSHIELD, AND WHAT IT TALKS TO



AgentShield is one box in the caller's payment chain: it is consulted at POST /v1/orders and returns one of three answers. Only ALLOW proceeds to the payments API. Everything on the right that is slow — webhooks, token events, language models — reaches the system through the asynchronous plane, never through the request the caller waits on.

Figure 1 — System context. The caller chain on the left reaches AgentShield at `POST /v1/orders`. AgentShield holds both planes internally and returns one answer; only allow reaches the payments API. Everything slow on the right — webhooks, token events, language models — enters through the asynchronous plane, never through the waiting request.

The boundary has exactly four kinds of traffic across it, and it is worth naming them precisely, because every later section is an elaboration of one of these four.

INBOUND · SYNC	One gRPC call, `Evaluate(OrderContext)`, on the order-creation path. This is the only thing the caller waits on.
OUTBOUND · SYNC	One answer, `Decision{ id, code, retryable }`. On `ALLOW` the caller continues to the payments API; on the other two it does not.
INBOUND · ASYNC	Razorpay `payment.*` webhooks, and the two token events (`token.confirmed`, `token.cancellation_initiated`). These feed the workers, never the request.
NEVER ON THE CLOCK	Any call to an LLM or an embedding model. These run only in the asynchronous plane, and their output reaches the request as a stored number.

> **WHY THE PLACEMENT IS THE DESIGN**
>
> Sitting at `POST /v1/orders` is not an implementation detail — it is the reason the system can be deterministic and honest. The order carries the amount and the mandate reference, so the hard questions ("is this within authority? does the amount bind?") can be answered by comparison, and the answer can be a cheap re-confirm because nothing has settled yet.

2 The component map

Everything that gets deployed, on one page — and the single store the two planes share.

The synchronous plane is deliberately small: one stateless Go service in front of three keyed stores. It holds no state it cannot rebuild. The asynchronous plane is a durable bus and six workers — Go where the work is stream-joining, Python where it is a model. The two planes are joined by exactly one shared store, the online feature store: the workers write it, the request reads it, and neither side ever calls the other.

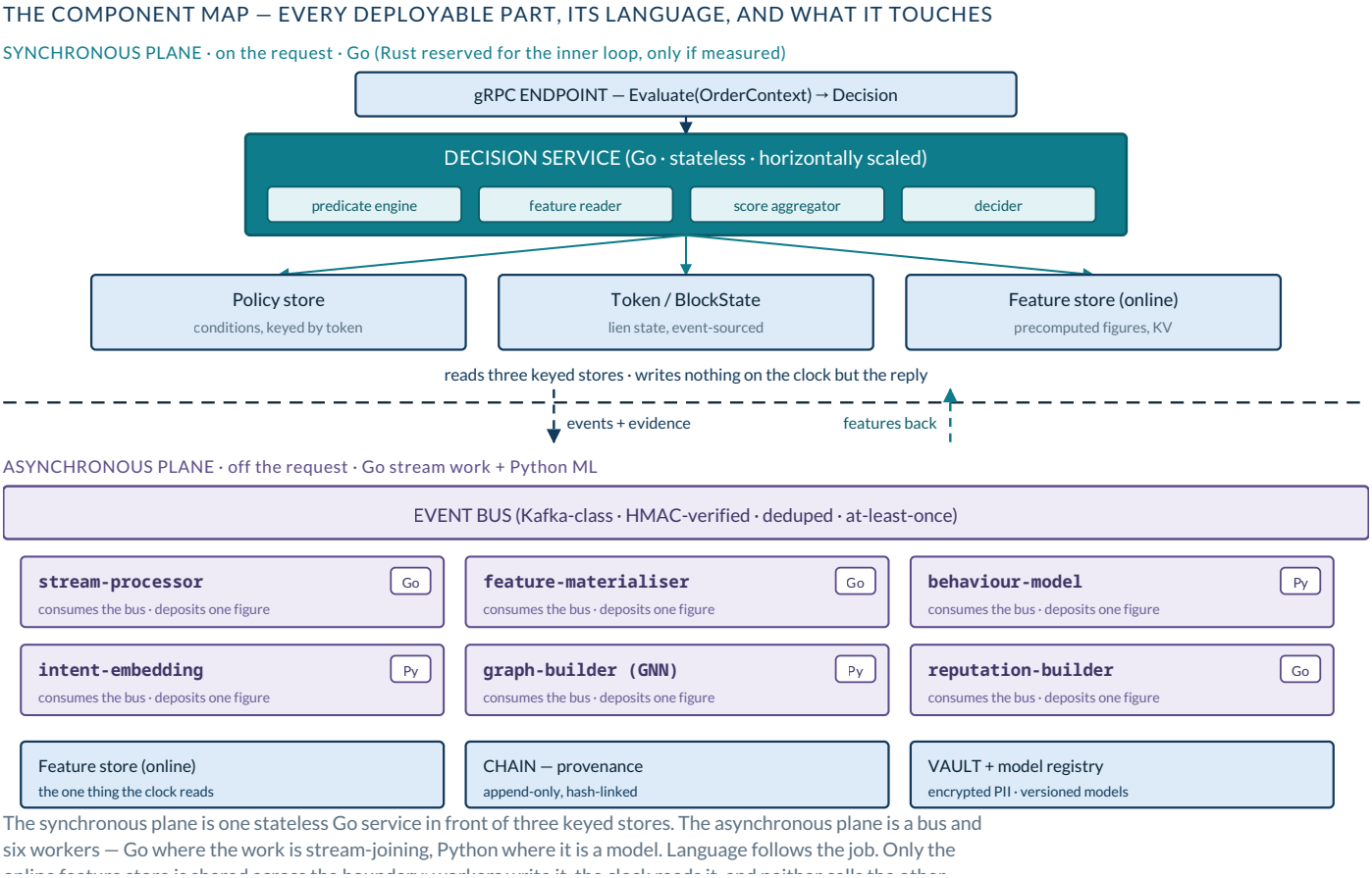


Figure 2 — The component map. Above the dashed line, the decision service and its three read stores. Below it, the bus and six workers, with the durable stores they own. The only wire that crosses is a write down to the bus and a read up from the feature store.

EVERY COMPONENT, ITS PLANE AND WHAT IT TOUCHES

COMPONENT	PLANE	LANG	RESPONSIBILITY	READS / WRITES
gRPC endpoint	sync	Go	Terminate `Evaluate`, authenticate the caller (mTLS)	—
Decision service	sync	Go	Orchestrate the seven stages; the only thing on the clock	reads policy, token, feature
predicate engine	sync	Go · Rust*	Run P1-P6, integer compares, terminal	reads token / block-state
feature reader	sync	Go	One keyed multi-get of the precomputed row	reads feature store
score aggregator	sync	Go · Rust*	In-process trees, expected-loss decide	—

COMPONENT	PLANE	LANG	RESPONSIBILITY	READS / WRITES
Event bus	async	Kafka	Durable, ordered per key, at-least-once	—
stream-processor	async	Go	Join events → block-state and per-principal baselines	bus → token, feature
feature-materialiser	async	Go	Windowed aggregates, point-in-time	bus → feature store
behaviour-model	async	Python	GBDT + isotonic + label-free floor	bus, registry → feature
intent-embedding	async	Python	Seal the envelope, embed, score drift	vault → feature
graph-builder (GNN)	async	Python	GraphSAGE embeddings, cluster / ring labels	bus → feature
reputation-builder	async	Go	Per-tool and per-agent trust accrues	bus → feature
CHAIN	store	SQL	Append-only, hash-linked provenance	written after reply
VAULT + registry	store	SQL	Encrypted PII; versioned model artifacts	off the clock

* Rust is reserved for the predicate-and-score inner loop and is adopted only if a measured p99 proves Go's garbage collector is the bottleneck — a decision moved on a number, not a hunch. Until then every component is Go or Python, as marked.

02

PART TWO

The synchronous plane

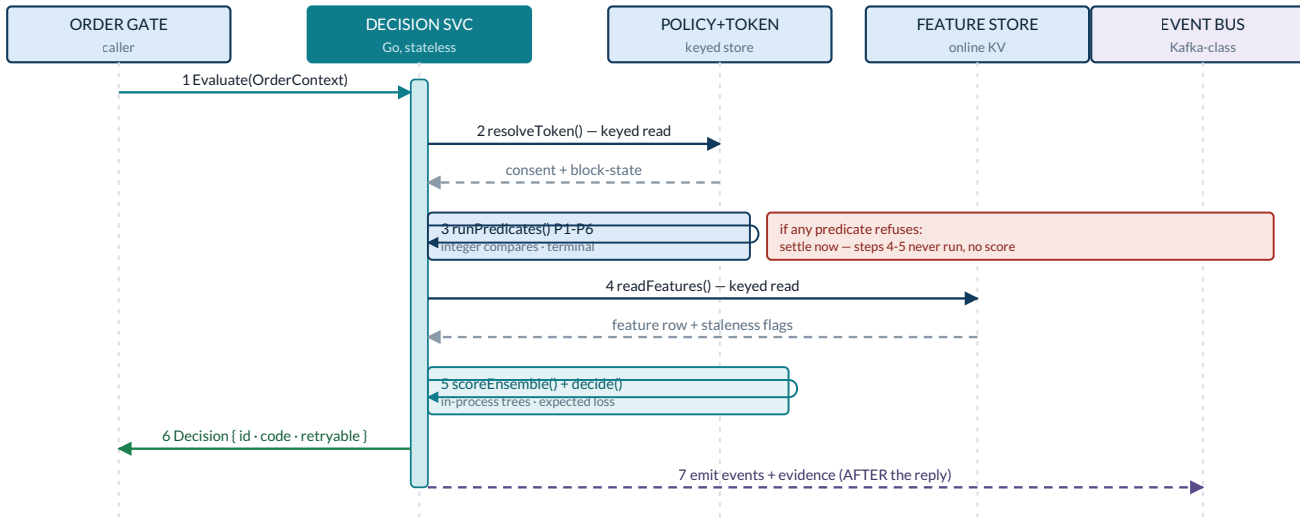
One stateless service in front of three keyed stores. It resolves a token, runs six integer checks, reads a row of precomputed figures, scores and decides — and replies before it tells the bus anything. This part names each stage, the contract at the edge, and the six predicates that are the only thing allowed to block.

3 The decision service

Seven stages, in one order, each a keyed read or an integer compare — and the reply leaves before the bus hears anything.

The decision service is a single stateless process. A request enters at the gRPC endpoint and passes through seven stages in a fixed order. The order is the design: the cheap, terminal checks run first, so the expensive stages never run for a request a comparison has already settled. Read the sequence below as a set of calls between the service and its stores — the caller waits only for message 6, and message 7 happens after the reply has left.

THE REQUEST, AS A SEQUENCE — WHO CALLS WHOM, IN ORDER, ON THE CLOCK



Read top to bottom. The caller waits only for message 6; message 7 is emitted after the reply has left, so the bus and the whole asynchronous plane are never on the caller's clock. Steps 4 and 5 are guarded — a refusing predicate at

Figure 3 — The request as a sequence. The decision service reads the policy / token store and, only if the predicates pass, the feature store; it replies to the caller, and only then emits events and evidence to the bus. Steps 4 and 5 are guarded by step 3.

THE SEVEN STAGES AND WHAT EACH OWNS

#	STAGE	WHAT IT OWNS	KIND	TARGET
1	`ingress`	Decode the request; authenticate the caller over mTLS	wire	~3 ms
2	`resolveToken()`	One keyed read of consent and block-state	keyed read	~6 ms
3	`runPredicates()`	Six integer checks, P1-P6; any one is terminal	compute	~2 ms
4	`readFeatures()`	One keyed multi-get of the precomputed figures	keyed read	~4 ms
5	`scoreEnsemble()`	In-process boosted trees → a calibrated probability	compute	~3 ms
6	`decide()`	Expected loss (p × ₹) against the interruption cost	compute	~1 ms
7	`respond()`	Reply first — *then* emit events to the bus	wire	~2 ms

What the service is, and is not, allowed to do

On this path there is no model call, no LLM call, and no third-party network hop of its own. Every box is a read from an in-memory store or arithmetic in the process. The one boosted-tree score in stage 5 is evaluated in-process from an exported model — microseconds, no model server. If a store the decision needs is slow or gone, the service does not wait and does not guess: it settles to a step-up and records the missing signal as missing.

THE ONE ORDERING RULE TO PRESERVE WHEN BUILDING

`runPredicates()` runs before `readFeatures()`, and a refusing predicate returns immediately. So when you add a feature to the score, you never change what can block, and you never make a rejected request pay for a feature read. Keep that boundary and the budget takes care of itself.

4 The Evaluate contract

One request in, two artifacts out — and the caller is told the decision and a code, never the score.

The synchronous plane exposes exactly one method, `Evaluate(OrderContext) → Decision`, over gRPC and Protobuf. The request carries everything the predicates and the score need, all of it by value or by key — there is nothing here the service has to go and fetch from a third party. The response splits in two: a lean answer to the caller, and a full record to the console and the provenance chain.

The request — `OrderContext`

FIELDS CARRIED ON THE REQUEST

FIELD	TYPE	MEANING & WHERE IT IS USED
`evaluation_id`	string	Idempotency key; dedupes retries — the input to P1
`token_id`	string	The mandate / permission slip; the key for policy and block-state
`customer_id`	string	The human principal; a feature-store key
`agent_id`	string	The acting agent; a feature-store and graph key
`merchant_id`	string	The payee; used by P2 scope and the graph
`session_id`	string	The session the intent envelope was sealed in
`amount_paise`	int64	Debit amount in paise; P2, P6 and the expected-loss decide
`cart_hash`	string	Digest binding the order to its line items — the input to P6
`envelope_digest`	string	Sealed intent digest from `order.notes`; P3 and intent
`tool_risk`	enum	`low|medium|high|critical`; a declared field, not a guess
`nonce · ts`	string · int64	Replay nonce and client timestamp; P1 and freshness

The response — two artifacts, deliberately unequal

To the caller. A four-field answer and nothing more. `evaluation_id`, a `decision` (`ALLOW` / `STEP-UP` / `BLOCK`), a fixed `code` from the enum below, and a `retryable` flag. There is no score, no band, no threshold — a caller who could see the score could probe for it. The code is a fixed enum, never an interpolated string.

To the console and CHAIN. The full record: `risk_score`, `risk_level`, the `predicate_failed` if any, the `signals` with their observation counts, the human-readable `reasons`, the `policy_version`, an `evidence_digest`, and the measured latency. This is what an operator and an auditor see — and it is written *after* the reply.

THE CALLER-FACING CODE ENUM (FIXED; EACH MAPS TO A CAUSE)

CODE	DECISION	CAUSE
`OK_ALLOW`	`ALLOW`	Expected loss below the interruption cost
`STEPUP_SCOPE`	`STEP-UP`	P2 — outside agreed scope
`STEPUP_UNBOUND`	`STEP-UP`	P3 — debit not bound to an instruction
`STEPUP_RISK`	`STEP-UP`	Score raised expected loss above cost

CODE	DECISION	CAUSE
STEPUP_FAILCLOSED	STEP-UP	A needed signal was missing, stale or errored
BLOCKED_DUPLICATE	BLOCK	P1 — replay of a seen request
BLOCKED_AUTHORITY	BLOCK	P4 — authority expired, revoked or exhausted
BLOCKED_IDENTITY	BLOCK	P5 — identity or slip cannot be verified
BLOCKED_BINDING	BLOCK	P6 — amount does not bind to the order

5 The six predicates

The deterministic spine. They need no labels and no model — and they are the only thing in the system that can block.

The six predicates are pure integer and set comparisons over data already in hand. They run first and alone: before any feature is read, before any score is computed. Any one that refuses settles the request immediately, and the reason is one of the fixed codes. This is where the product's core question — *was this payment asked for?* — is answered without probability, and it is the reason the system has something honest to say on day one, before a single model has trained.

P1 THROUGH P6 — INPUTS AND VERDICT

PREDICATE	ASKS	INPUTS	ON FAILURE
P1 · Replay	Have I already seen this exact request?	`evaluation_id, nonce, block-state`	`BLOCK` — a duplicate is never real money
P2 · Scope	Is it bigger than allowed, or to the wrong kind of place?	`amount_paise, merchant_id, policy caps + categories`	`STEP-UP` — allowed in principle, outside what was agreed
P3 · Unbound	Is this debit backed by the instruction we were given?	`envelope_digest, amount vs envelope max`	`STEP-UP` above a value floor; no envelope → unattested
P4 · Stale authority	Has the permission expired, been cancelled, or run out?	`expire_at, token status, consumed vs ceiling`	`BLOCK` — the authority no longer exists
P5 · Unverifiable	Can I prove who this is, and that the slip is theirs?	`caller identity, token binding`	`BLOCK` — no proof, no money
P6 · Binding	Does the amount asked match the amount on this order?	`amount_paise vs cart_hash`	`BLOCK` — the numbers must agree

Everything that must be *exact and current* — money limits, expiry, duplicates, identity, the amount binding — is checked live, on the clock, against the real request. None of it is trusted to a background figure. Notice the split in the verdicts: P1, P4, P5 and P6 `BLOCK`, because a failure there means the request is not real or not permitted; P2 and P3 `STEP-UP`, because the request may be genuine but has strayed from what was agreed.

ONE FIELD THAT LOOKS LIKE A PREDICATE BUT IS NOT A BLOCK

`tool_risk == critical` raises a `STEP-UP`, not a `BLOCK`. Tool risk is a *declared* field on the request, not something we verified — so it can heighten caution and ask a human, but it must never refuse on its own. Treating a declared value as grounds to block would hand any caller a way to force outcomes.

WHY THIS IS A SPINE, NOT A MODEL

Predicates are cheap, explainable and stable: the same inputs always give the same verdict, and every verdict has a one-line reason. When we build the ML engines in Part five, they sit *on top of* this spine — they can raise risk and ask for a step-up, but they can never reach past it to block or to override a block. Build the spine first; everything else is an addition to it.

03

PART THREE

The asynchronous plane

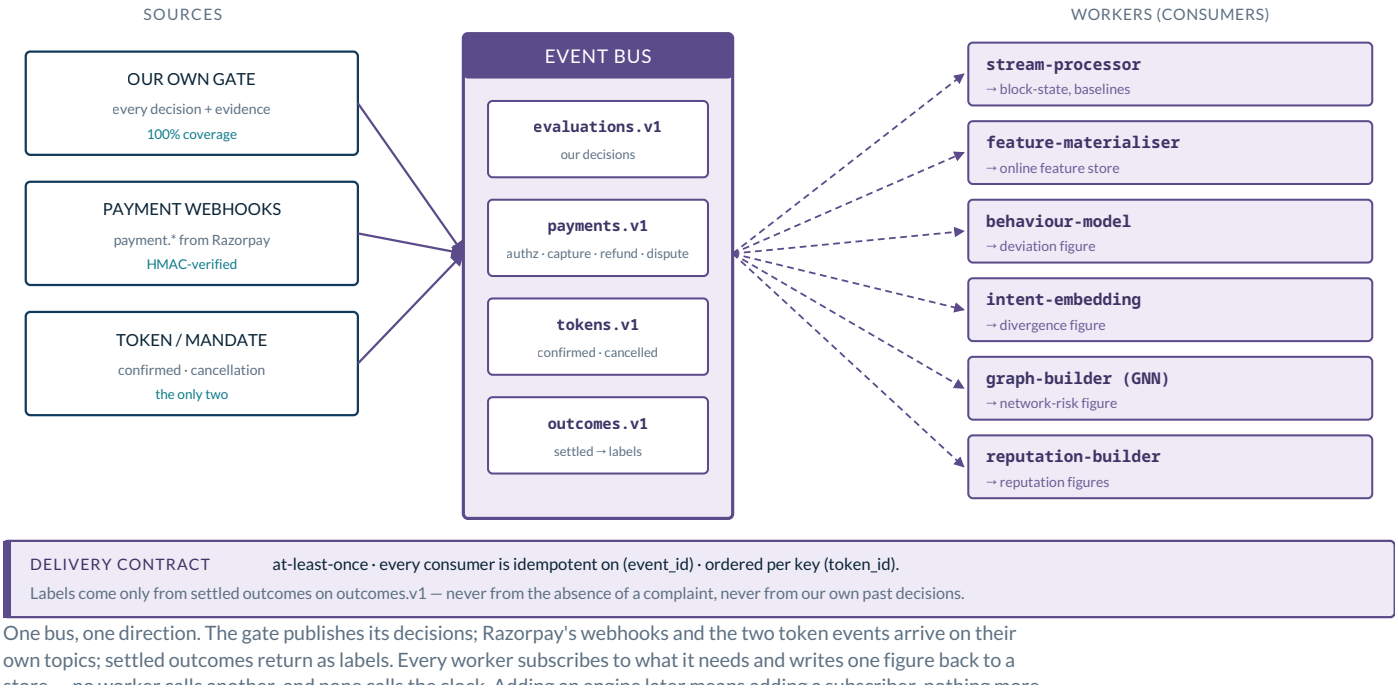
A bus and six workers. They consume the gate's own decisions, Razorpay's webhooks and the two token events, recompute baselines, models, embeddings and reputation, and leave each result in a store keyed for one lookup. This part names the topics, the workers, and the one-directional contract that lets a slow model feed a fast path without ever being on it.

6 The event bus

Three sources in, four topics, six workers out — and it runs one way, always.

The asynchronous plane is fed by one durable bus. Three sources publish to it, four topics carry the traffic, and the six workers subscribe to what each needs. The bus is the seam of the whole system: it is how the fast path tells the slow path what happened, and how the world tells both. Crucially, it runs in one direction — the clock publishes and walks away; nothing on the bus ever calls back into the clock.

THE EVENT BUS — THREE SOURCES IN, FOUR TOPICS, SIX WORKERS OUT



One bus, one direction. The gate publishes its decisions; Razorpay's webhooks and the two token events arrive on their own topics; settled outcomes return as labels. Every worker subscribes to what it needs and writes one figure back to a store — no worker calls another, and none calls the clock. Adding an engine later means adding a subscriber, nothing more.

Figure 4 — The event bus. Sources on the left, topics in the centre, workers on the right. Every worker writes one figure to a store; none calls another worker, and none calls the request. Adding an engine later is adding a subscriber.

The three sources

- Our own gate. Every evaluation the decision service makes is published, with its evidence — 100% coverage of what we decided. This is the spine of the learning loop.
- Razorpay payment webhooks. The `payment.*` lifecycle — authorised, captured, failed, refunded, disputed — HMAC-verified at the edge before it is trusted.
- Token / mandate events. Exactly two: `token.confirmed` and `token.cancellation_initiated`. There is no per-debit token event — block-state is reconstructed from these two plus our own payment records, never assumed.

The four topics

TOPICS AND WHAT THEY CARRY

TOPIC	CARRIES	KEY
`evaluations.v1`	Every decision + evidence digest from the gate	`token_id`
`payments.v1`	Authorised, captured, failed, refunded, disputed	`token_id`
`tokens.v1`	Confirmed and cancellation-initiated, only	`token_id`

TOPIC	CARRIES	KEY
`outcomes.v1`	Settled outcomes, turned into training labels	`token_id`

THE DELIVERY CONTRACT EVERY CONSUMER MUST HONOUR

Delivery is at-least-once, so every worker is idempotent on **`event_id`** — processing the same event twice must not double-count. Ordering is guaranteed per key (`token_id`), which is why every topic is keyed the same way: a token's events never arrive out of order relative to each other. Version the schema in the topic name (`.v1`) so a breaking change is a new topic, not a silent reinterpretation.

WHERE LABELS ARE ALLOWED TO COME FROM

A training label may come only from a settled outcome on `outcomes.v1` — a dispute, a chargeback, a cancellation, a confirmed step-up. Never from "no complaint arrived," and never from our own past decisions. Learning from our own outputs would teach the models to agree with yesterday's mistakes.

7 The workers

Six subscribers, each doing one job and leaving one number behind — and never talking to each other.

Every worker follows the same shape: subscribe to the topics it needs, do its one job, and deposit the result into a store keyed for a single lookup. None of them calls another worker, and none reaches into the request path. That independence is what lets us build them one at a time, in phase order, and add a seventh later without touching the six.

THE SIX WORKERS

WORKER	CONSUMES	COMPUTES	CADENCE	DEPOSITS
`stream-processor`	evaluations, payments, tokens	Join events → block-state + per-principal baselines	continuous	block-state, baselines
`feature-materialiser`	all topics	Windowed, point-in-time aggregates	continuous	feature-store rows
`behaviour-model`	baselines, outcomes	GBDT + isotonic + label-free floor	retrain daily-weekly; score continuous	`behaviour_deviation`
`intent-embedding`	session start, evaluations	Seal envelope once, embed, per-debit drift	per session + per debit	`intent_divergence`
`graph-builder`	all topics (edges)	GraphSAGE embeddings, cluster / ring labels, propagation	hourly-daily	`network_risk`
`reputation-builder`	evaluations, outcomes	Per-tool and per-agent trust accrual	continuous	`reputation`

Why the language is split by worker

The first two workers are stream-joins over high-volume events — steady throughput, predictable pauses, no model — so they are Go, the platform language. The next three carry a model, an embedding or a graph network, where the ecosystem matters, so they are Python. Reputation is bookkeeping and stays in Go. The split is not aesthetic: it puts each job in the runtime that fits it and keeps the ML libraries out of the stream path.

> THE BUILD ORDER THESE WORKERS IMPLY
>
> Because the workers are independent, they map cleanly onto phases: stream-processor and feature-materialiser come first (they feed everything), then behaviour, then intent, then graph, then reputation. Each new worker is a new subscriber and a new column in the feature row — the request keeps reading the same way while the shelf behind it fills in.

8 The coupling and staleness

The two planes meet at exactly two places, and neither ever waits on the other. This is the contract that makes the design safe.

If there is one rule to hold in mind while building anything that spans the planes, it is this: the coupling is one-directional. The clock never calls a worker. No worker calls the clock. They communicate only through durable infrastructure, at two well-defined meeting points.

MEETING POINT 1	Down to the bus. After it has replied, the decision service emits its evaluation and evidence. This is a fire-and-forget publish — if the bus is briefly unavailable, the reply has already gone out, and the event is buffered and retried.
MEETING POINT 2	Up from the feature store. On the next request, the service reads whatever figures the workers have deposited — by key, in single-digit milliseconds. It reads the current value; it never asks a worker to compute a fresh one.

Everything else follows. A worker can be slow, restarting, or mid-retrain and the request still returns, because the request never depended on the worker being up — only on the last figure it left behind. This is the property that lets a slow model feed a fast path without ever being on it.

Staleness is a first-class fact, not an accident

Because the request reads a value a worker wrote earlier, that value has an age. The system treats age as data, not as something to hide. Every figure in the feature store carries a `computed_at` timestamp, and the decision service reads it alongside the value.

THE THREE STALENESS CASES, AND WHAT EACH DOES

Fresh. Used as is. Lagging — a figure older than its freshness target: it is *used and flagged*, and the flag is part of the evidence. Missing — no figure at all: it is treated as missing, which raises risk. A blank is never filled with an optimistic zero, because "no signal" and "a signal that says all-clear" are not the same thing.

FAIL-CLOSED, STATED ONCE FOR THE WHOLE SYSTEM

When any figure the decision needs is missing or stale past tolerance, or a store read errors or times out, the request does not guess to stay fast and does not hard-decline on a blank. It settles to `STEP-UP` with the code `STEPUP_FAILCLOSED`, and records exactly which signal was absent. On the order-creation gate a step-up costs nothing on the rail — asking again is cheap; a fast wrong yes is not.

04

PART FOUR

The data design

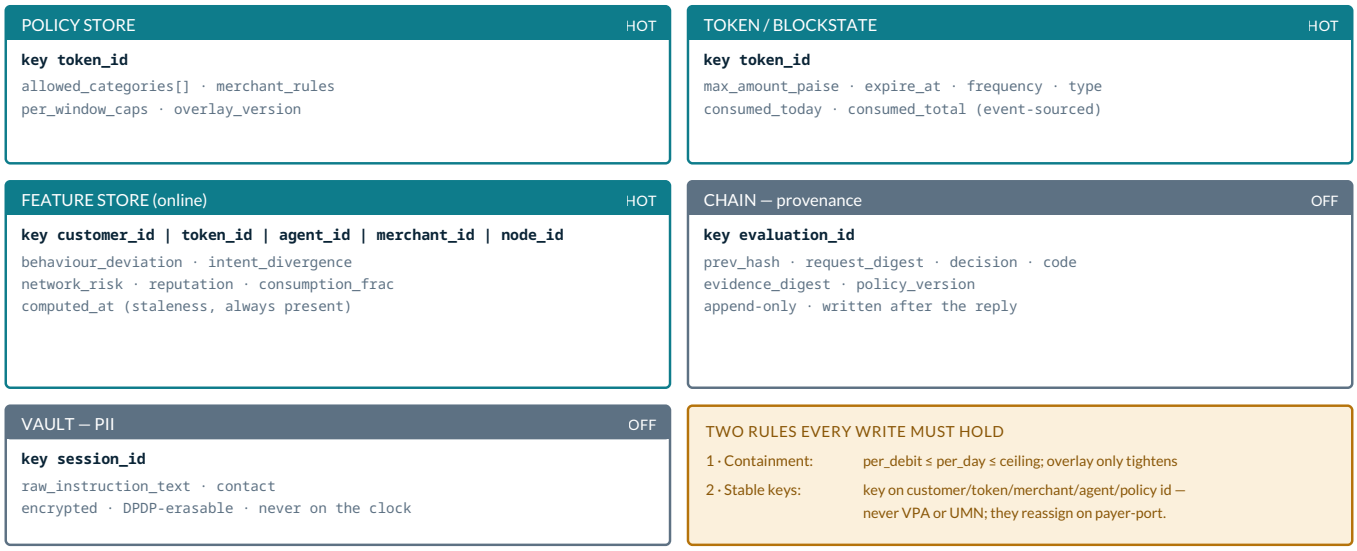
Five stores, and the shape of everything that moves between them. Three stores are on the clock and reached by a single key; two never touch the request. This part fixes the key for each store and writes down every core schema — token, policy overlay, intent envelope, feature row, provenance record and event — the way it is actually stored.

9 The five stores

Three on the clock, two off it — and every one reached by a single key.

The data plane is five stores. Three are read on the request and are chosen for one property above all: a keyed read returns in single-digit milliseconds, with no scan and no join. The joins already happened off the clock, when the feature row was materialised and the block-state was reconstructed from the event stream. Two stores never touch the request at all — the provenance chain and the PII vault.

THE DATA MODEL — WHAT EACH STORE HOLDS, AND THE KEY IT IS REACHED BY



Five stores, three of them on the clock. Everything the request reads is reachable by a single key — no scan, no join — because the joins already happened off the clock when the feature row and the block-state were built. The feature store always carries computed_at, so a stale figure is used and flagged and a missing one is treated as missing, never as zero.

Figure 5 — The data model. Each store with the key it is reached by and the shape it holds. The three hot stores return by a single key; the feature store always carries `computed_at`. The two rules on the right hold for every write.

THE FIVE STORES AND HOW THEY ARE REACHED

STORE	ON CLOCK	KEY	ACCESS PATTERN	TECHNOLOGY
Policy store	hot	`token_id`	Keyed read of the customer's conditions	in-memory KV; SQL source of truth
Token / BlockState	hot	`token_id`	Keyed read; reconstructed from token events + our payments	in-memory KV, event-sourced
Feature store	hot	`customer / token / agent / merchant / node`	Keyed multi-get of precomputed figures, point-in-time	Redis / Aerospike / Dragonfly
CHAIN	off	`evaluation_id`	Append-only, hash-linked; written after the reply	append log / SQL, never deleted
VAULT	off	`session_id`	Never read on the hot path; instruction text, contact	encrypted SQL, DPDP-erasable

THE AVAILABILITY THAT ACTUALLY MATTERS

It is the policy and block-state stores', not the scorer's. Predicates run without the model; nothing runs without the boundary. So when you plan replication and failover, replicate the boundary stores first — a request can survive a cold feature store (it fails closed to a step-up), but it cannot survive not knowing the token's limits.

TWO KEYS THE WHOLE DESIGN REFUSES TO USE

Nodes and rows are keyed on stable identifiers — `customer_id`, `token_id`, `merchant_id`, `agent_id`, `policy_id`. Never on VPA or UMN: those reassign on a payer-port, so a figure keyed on them would silently attach one entity's history to another. This rule matters most in the graph, where a mis-key is a mis-identification.

10 The core schemas

Six objects carry the whole system. Here is the shape of each, the way it is actually stored.

Everything that moves between components is one of six objects. They are small on purpose. Amounts are always in paise (₹2,000 is `200000`) so there is no floating-point money anywhere, and identifiers are the stable keys of Section 9.

Token — the permission slip

FIELD	TYPE	MEANING
`token_id`	string	The mandate; primary key everywhere
`customer_id`	string	The human principal it acts for
`type`	enum	`recurring | one_time | top_up`
`max_amount_paise`	int64	Per-debit ceiling
`max_per_day_paise`	int64	Daily ceiling
`token_ceiling_paise`	int64	Lifetime ceiling of the mandate
`frequency`	string	Permitted cadence (daily / weekly / monthly)
`expire_at`	int64	Epoch expiry; P4 compares against it
`status`	enum	`pending | confirmed | cancelled`

Containment invariant: `max_amount_paise ≤ max_per_day_paise ≤ token_ceiling_paise`, and a policy's expiry must be ≤ the token's. A write that violates this is rejected, not clamped.

Policy overlay — the customer's tightening

FIELD	TYPE	MEANING
`token_id`	string	The token this overlay tightens
`allowed_categories`	[]string	Merchant categories permitted
`merchant_rules`	object	Allow / deny lists by merchant
`per_window_caps`	object	Caps tighter than the token's
`overlay_version`	int	Monotonic; recorded in every decision

Rule: an overlay can only *tighten*, never widen. A write that would raise a cap or admit a category the token forbids is rejected. P2 reads the effective (token ∩ overlay) bound.

Intent envelope — sealed once per session

FIELD	TYPE	MEANING
`session_id`	string	The session it was captured in
`purpose`	string	What the user asked for, in words

FIELD	TYPE	MEANING
`category`	string	Merchant category expected
`max_amount_paise`	int64	The ceiling the user stated
`merchant_preference`	string	Named or preferred payee, if any
`constraints`	[]object	Explicit conditions (once, before date…)
`envelope_digest`	string	Sealed hash — written to `order.notes`

The digest travels on the request; the raw text lives in the VAULT, keyed by `session_id`, and is erasable. The LLM that builds this runs once, at session start, off the clock — it never sees a policy, a score or a threshold, and it never decides.

Feature row — what the request reads

FIELD	TYPE	MEANING
`key`	string	The entity id (customer / token / agent / merchant / node)
`behaviour_deviation`	float	Calibrated deviation figure from the behaviour engine
`signal_deviations`	[]object	Per-signal deviation *with its observation count*
`intent_divergence`	float	Judged distance from the sealed envelope
`network_risk`	float	Calibrated figure from the graph engine
`reputation`	float	Per-agent / per-tool trust
`consumption_frac`	float	Fraction of the block consumed — model-free day-one signal
`computed_at`	int64	Freshness stamp — always present, drives staleness handling

The observation counts matter: a deviation seen over three events is not the deviation seen over three thousand, and the aggregator shrinks a thin signal toward its prior rather than trusting it.

Provenance record — one row per decision on the CHAIN

FIELD	TYPE	MEANING
`evaluation_id`	string	Primary key; the same id the caller received
`prev_hash`	string	Hash of the previous record — the chain link
`request_digest`	string	Digest of the `OrderContext` seen
`decision · code`	enum	The answer and its fixed code
`predicate_failed`	enum?	Which of P1-P6 refused, if any
`evidence_digest`	string	Digest of the signals and reasons
`policy_version`	int	The overlay version in force
`ts`	int64	When it was written — after the reply

Event envelope — every message on the bus

FIELD	TYPE	MEANING
`event_id`	string	Idempotency key — consumers dedupe on it
`type`	string	e.g. `payment.captured` , `token.confirmed`
`token_id`	string	Partition key — guarantees per-token ordering
`occurred_at`	int64	Event time, for point-in-time joins
`payload`	object	Type-specific body
`source · hmac`	string	Origin, and the signature verified for webhooks

> **WHY THIS LIST IS CLOSED**
>
> Six objects, and no request touches anything outside them. New engines add *columns* to the feature row and *types* to the event envelope — not new objects on the hot path. Keeping this list closed is what keeps the request a set of keyed reads.

05

PART FIVE

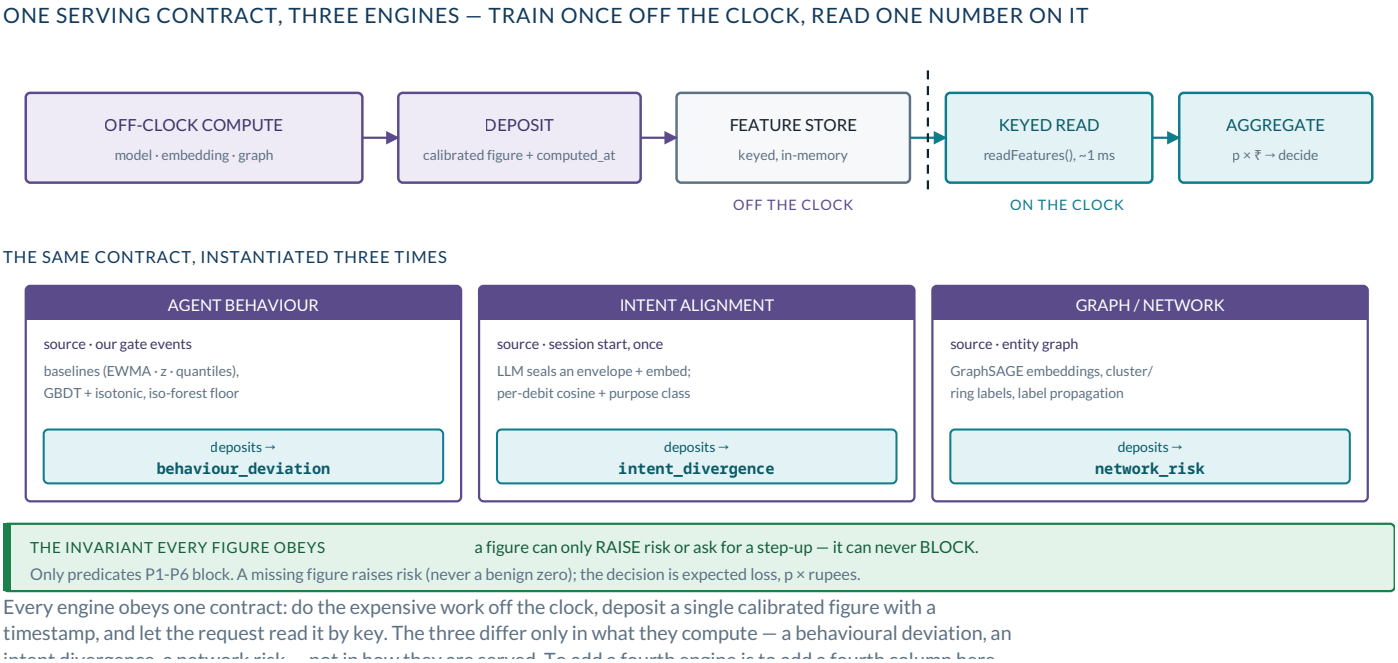
The three ML engines as services

The priority of the whole build, and they share one serving contract: train off the clock, deposit a single calibrated figure with a timestamp, and let the request read it by key. This part designs each — behaviour, intent alignment, graph — as the service it is, and shows why none of them is ever on the request.

11 The behaviour engine

A streaming pipeline that learns each principal's normal — and leaves one calibrated number on the shelf.

All three ML engines obey one serving contract, so it is worth seeing it once before the first engine. Each does its expensive work off the clock, deposits a single calibrated figure with a `computed_at` stamp, and lets the request read it by key. The engines differ only in *what* they compute — not in how they are served.



Every engine obeys one contract: do the expensive work off the clock, deposit a single calibrated figure with a timestamp, and let the request read it by key. The three differ only in what they compute — a behavioural deviation, an

Figure 6 — One serving contract, three engines. The generic flow across the top; the three engines instantiating it below. A figure only ever raises risk or asks for a step-up — it can never block, and a missing figure raises risk rather than defaulting to zero.

The behaviour engine answers one question: *is this principal acting unlike itself?* It is a three-layer stack, and every layer runs in the asynchronous plane.

Layer 1 — per-principal streaming baselines

For each `customer_id`, `token_id` and `agent_id`, the stream-processor maintains a running sense of normal: EWMA and robust z-scores on amount, rolling quantiles, velocity, and sketch-based distinct counts (HyperLogLog for distinct merchants, Count-Min for frequencies) so memory stays bounded at any volume. On cold start — a principal we have barely seen — the baseline is shrunk toward a peer or population prior rather than trusted thin.

Layer 2 — the calibrated scorer

A gradient-boosted-tree model (LightGBM / XGBoost) turns the baseline deviations into a score, and isotonic calibration turns that score into a probability. Calibration is not optional here: the aggregator on the clock multiplies this figure by rupees to get an expected loss, so it must be a real probability, not a rank.

Layer 3 — the label-free floor

Beneath the supervised model sits an unsupervised floor — an isolation forest now, an autoencoder later — so that a pattern never seen in the labels still registers as anomalous. And beneath even that is the one signal that needs no model at all: the block-consumption fraction and rate, which is meaningful from the very first debit.

WHAT IT DEPOSITS

One calibrated `behaviour_deviation`, plus the per-signal deviations *each with its observation count*, and the consumption figure. The counts let the aggregator shrink a thin signal toward its prior. Baselines update continuously; the trees retrain daily to weekly. Nothing here is ever computed on the request — the request reads the number.

12 The intent-alignment engine

Understand the instruction once; then every debit is a cheap comparison against a sealed fingerprint.

Intent alignment sounds like something you can only judge in the moment — *does this payment match what the user actually asked for?* The engine splits it into a slow part done once and a cheap part done every debit, and neither is on the clock.

Once, at session start — seal the envelope

Before any tool call, an LLM reads the user's instruction and extracts a structured intent envelope: `{ purpose, category, max_amount_paise, merchant_preference, constraints }`. A sentence-transformer (bge-small or all-MiniLM-L6-v2) embeds it, and the whole thing is sealed: the `envelope_digest` is written to `order.notes` so it travels on every later request, and the raw text goes to the erasable VAULT. This is the only place a language model runs, and it runs exactly once.

Every debit — measure divergence in two parts

When a debit arrives, alignment is a comparison against that sealed envelope, and it divides cleanly:

The exact part. Is the amount within the envelope's stated maximum? Are the explicit constraints satisfied? These are crisp comparisons — so they are handled deterministically, as predicate P3 and policy, not by a model. A hard breach here is a step-up on the clock, not a soft score.

The judged part. How close is this merchant and category to what was asked? This is cosine similarity on the embeddings plus a small purpose classifier (purchase vs subscription vs top-up). It produces the `intent_divergence` figure — a soft distance, never a verdict.

> **NO ENVELOPE MEANS UNATTESTED, NOT INNOCENT**
>
> If a session produced no envelope, the debit is unattested — it is *not* scored as aligned, and it is *not* quietly passed. The absence is recorded and it raises risk, exactly as a missing figure does elsewhere. Silence is never read as consent.

> **THE LINE THE LLM NEVER CROSSES**
>
> The language model extracts and embeds. It never sees a policy, a score or a threshold, and it never decides. Its output becomes a *feature* — the sealed envelope and the divergence figure — and a feature can only raise risk or ask for a step-up. An LLM's opinion is an input to the decision, never the decision.

13 The graph engine

Some risk is only visible in the shape of the neighbourhood — so we learn the neighbourhood off the clock and read one number on it.

The behaviour engine asks whether a principal is acting unlike itself; the graph engine asks a question no single-principal view can answer: *is this principal embedded in a structure that looks like fraud?* Collusion rings, mule fan-out, a fresh agent that shares a device with twenty settled-bad tokens — these are properties of a neighbourhood, not of a row. The engine learns that neighbourhood off the clock and leaves a single calibrated `network_risk` on the shelf.

The graph — heterogeneous, time-aware, inductive

Nodes are the stable entities — `customer`, `token`, `merchant`, `agent`, `policy`, device `node` — and edges are the debits and shared attributes that tie them. Edges carry time, so the model sees a ring forming rather than a static snapshot, and the architecture is inductive (GraphSAGE-style, via PyG or DGL): a node that appeared this morning gets an embedding from its neighbourhood without a full retrain.

Learning without many labels

Fraud labels are scarce and late, so the engine does not lean on them alone. It computes inductive node embeddings, runs cluster / ring detection over them, and uses label propagation from the few settled-fraud seeds to spread suspicion through tightly connected structure. The supervised signal sharpens what the structure already suggests — it is not the only thing holding the figure up.

The structural floor

Beneath the learned embedding sits a model-free floor, exactly as the behaviour engine has one: a node with no learned embedding yet still has a structural signal — degree, fan-out, count of shared attributes, size of the component it sits in. It is meaningful from the first sighting, so a brand-new node is never simply blank.

> **WHAT IT DEPOSITS, AND WHAT THE CLOCK DOES**
>
> The graph is built, the embeddings are computed and the propagation runs entirely in the asynchronous plane; the engine deposits one calibrated `network_risk` per node with a `computed_at` stamp. The request does no graph walk — it reads the node's figure by key, in single-digit milliseconds, like every other feature. The graph rebuilds and re-embeds on a batch cadence; the request only ever reads.

> **A MIS-KEY HERE IS A MIS-IDENTIFICATION**
>
> Every node is keyed on a stable identifier — never on VPA or UMN, which reassign on a payer-port. This rule from Section 9 matters most in the graph: a figure keyed on a reassignable handle would silently attach one entity's history to another, and in a graph that is not a stale number — it is the wrong person's whole neighbourhood.

06

PART SIX

Building from this design

The concerns that cut across every component — cold start, fail-closed, idempotency, caller authentication, observability — and then the table this whole document exists to produce: from a feature we are about to build, to the components, stores, contracts and phase it touches.

14 Cross-cutting concerns

Five properties that belong to no single component because they belong to all of them. Get them wrong once and every engine inherits the bug.

The parts have been drawn one at a time, but a few concerns run through every one of them. They are stated here once, so that no component has to reinvent them and no two components disagree.

COLD START	A principal or node we have barely seen is never trusted thin and never blanked to zero. The behaviour engine shrinks toward a peer or population prior, the graph engine falls back to its structural floor, and a session with no envelope is unattested. Thinness raises risk; it does not earn a pass.
FAIL-CLOSED	Stated in full in Section 8: any figure the decision needs that is missing, stale past tolerance, or behind a store that errors settles the request to `STEP-UP` with `STEPUP_FAILCLOSED` — never an optimistic allow, never a hard decline on a blank.
IDEMPOTENCY	Every bus message carries an `event_id` and consumers dedupe on it, so a redelivery changes nothing. On the clock, a replayed order is caught by predicate P1 and blocked; the `evaluation_id` the caller holds is the stable key for the one decision that order ever gets.
CALLER AUTHENTICATION	The gate is a network-exposed gRPC endpoint, so the caller is authenticated by mutual TLS before any evaluation runs — an unauthenticated caller never reaches a predicate. The reply carries a fixed code and no score or threshold, so a caller cannot probe the boundary by watching numbers move.
OBSERVABILITY	Every decision is written to the append-only, hash-linked CHAIN, and the service exports the four figures that tell you it is healthy: p99 latency, fail-closed rate, per-feature staleness rate, and the decision mix (allow / step-up / block).

THE ENDPOINT IS A BOUNDARY, SO IT IS AUTHENTICATED FIRST

Because the decision service sits in front of money, an open port is not an option: the caller is verified by mTLS, the request is size- and schema-bounded before it is parsed, and the two-artifact response keeps the score off the wire. Auth is not a feature to add later — a boundary without it is not a boundary.

WHAT "HEALTHY" MEANS, NUMERICALLY

The SLO that matters is on the boundary path: p99 ≤ 50 ms for a decision, with the typical path near 21 ms. But the availability target is placed on the policy and block-state stores, not on the scorer — a request survives a cold feature store by failing closed, but it cannot survive not knowing a token's limits. Watch the fail-closed rate: a spike there is the first sign a store, not the model, is in trouble.

15 The build map

The table this whole document exists to produce — from a feature about to be built, to the parts, stores and contracts it touches.

Everything so far named the parts. This section is the payoff: an order to build them in, chosen so that the boundary holds before any model exists, and so each engine is added as a figure that can only raise risk. The spine ships first and alone; the models arrive one at a time; the aggregator and the learning loop come last, when there is something to aggregate and something to learn from.

THE EIGHT PHASES, AND WHAT EACH ONE TOUCHES

PHASE	WHAT SHIPS	COMPONENTS & STORES IT TOUCHES	DONE WHEN
0	Contracts & skeleton	Evaluate gRPC contract, the six schemas, decision-service stub	a stubbed Evaluate returns a well-formed two-artifact reply over mTLS
1	Synthetic data & plumbing	event bus, the five stores, stream-processor + feature-materialiser skeletons	synthetic events flow on the bus and materialise feature rows with `computed_at`
2	The deterministic spine	P1-P6, `resolveToken()`, policy + token/block-state stores, `decide()`	the boundary blocks and step-ups correctly with no model in the loop
3	Behaviour engine	behaviour-model worker, feature store; deposits `behaviour_deviation`	a calibrated deviation is deposited and read by key on the clock
4	Intent engine	intent-embedding worker, VAULT, feature store; deposits `intent_divergence`	an unattested session raises risk; an aligned debit does not
5	Graph engine	graph-builder worker, feature store; deposits `network_risk`	`network_risk` is deposited per node and read by key — no walk on the clock
6	Ensemble, reputation & loop	`scoreEnsemble()`, reputation-builder, CHAIN, feature store	the aggregator turns the deposited figures into one calibrated decision; the loop retrains from settled labels
7	CLI demo & eval	the scenario CLI end-to-end, an evaluation notebook over synthetic ground truth	scripted scenarios show allow / step-up / block; the notebook reports decision quality

HOW TO USE THIS DOCUMENT WHILE BUILDING

Pick the feature you are about to build and read across: Section 2 names the component and its language, Section 4 names its contract, Sections 9–10 name the stores and schema it touches, and Sections 11–13 name the engine if it is a learned one. The phase above tells you what must already exist before you start.

THE ORDER IS A SAFETY PROPERTY, NOT A PREFERENCE

Phase 2 exists before Phase 3 on purpose: the deterministic spine must block and step-up correctly *with no model present*, because a model can only ever raise risk and never blocks. If the boundary is not trustworthy on its own, no amount of learned figure makes it so — so it ships first, and everything else is added on top of a boundary that already holds.

— The design, in one breath

Fifteen sections, but one shape — and it is worth holding that shape in mind as the code gets written.

A request is a set of keyed reads and integer compares that ends in a verdict, and it is fast because every slow thing already happened somewhere else. That is the whole design. Two planes that meet at exactly two places; a deterministic spine that can block and holds on its own; three learned engines that run off the clock and can only raise risk or ask again; six small objects that carry everything between them; and a boundary that, when it is unsure, fails closed to a step-up rather than guessing.

Nothing in this document asks a builder to trust a model with the money. The predicates decide what is refused; the engines only colour how much to worry. When you are deep in one component and the larger picture blurs, that division is the thing to recover: the boundary is deterministic, the intelligence is advisory, and silence is never read as consent.

> WHAT TO BUILD FIRST, AND WHY IT IS ENOUGH TO START
>
> Build Phase 0 through Phase 2 and you have a system that is already safe — it refuses the right requests with no model in the loop. Everything after that makes it *smarter*, not *safer*. That ordering is the quiet promise of this design: the risk layer earns its place before the first figure is ever deposited, and every engine added afterward is a feature on top of a boundary that was already correct.